Ghost Layer: A Cyber Immune Architecture for Endpoint Defense with Autonomous Label Generation*

Keerthivahanan [Surname]
Architecture and Detection Research
Ghost Layer Technologies
Chennai, India

Deepaksanjay [Surname]
Detection Engineering
Ghost Layer
Technologies Chennai,
India

Dhakshin [Surname]
Systems and Kernel Engineering
Ghost Layer Technologies
Chennai, India

Narine [Surname]
Interface and Analytics
Engineering
Ghost Layer Technologies
Chennai, India

Abstract Small and medium enterprises are attacked by the same people who attack large organizations, but they have no security team to run the tools built for them. Most EDR platforms make this harder by design. Detection, provenance analysis, deception and learning run as separate modules that meet only at an alert queue, so no module can improve another while an attack is still underway. We built Ghost Layer to test a different arrangement. Behavioral detection picks the seeds for causal analysis, the causal graph decides where decoys go, decoy interactions become training labels, and those labels feed back into the behavioral model. We call these four connections E1 through E4. E3 matters most: nothing legitimate touches a decoy, so an interaction is a positive label that no analyst had to approve. The artificial immune systems literature calls this self-generated recognition; we apply it to kernel telemetry rather than network traffic. The agent collects events through eBPF on Linux and Event Tracing for Windows, stores them in an embedded columnar database, and runs inference locally. We ran nine multi-stage attack scenarios against 18.2 million events of benign background activity. All nine were detected, four of them by the deception layer, with no false positives. We report the resource measurements we have, say plainly which ones we do not, and set out the ablation that remains to be run.

Keywords—endpoint detection and response, cybersecurity, eBPF, behavioral anomaly detection, causal analysis, deception technology, artificial immune systems, autonomous labeling, adaptive defense

I. INTRODUCTION

An organization with two hundred endpoints is attacked by the same people who attack one with twenty thousand. Ransomware affiliates, phishing kits and living-off-the-land tradecraft do not check headcount first. What the smaller organization does not have is a security team. It has one IT person, a contractor, or nobody. Every commercial EDR platform we looked at assumes otherwise: that someone is reading the alert queue, and that there is compute to spare for a cloud analytics tier. Neither assumption holds here.

We think the obstacle is architectural before it is commercial. Current platforms split detection into modules that run side by side and meet only at the alert layer. A behavioral engine, a deception subsystem, a provenance builder and a threat feed all read the same telemetry, all emit alerts, and a correlation stage tries to reassemble the story afterwards. None of them reads the others. Because each has to stand alone, each has to be tuned for precision alone, and whatever false positives survive that tuning are added together and handed to a person. If there is no person, the only workable setting is to raise every threshold, which is exactly the setting a patient attacker wants.

None of the individual pieces are missing. Provenance reconstruction works, as SLEUTH [3] and HOLMES [4] showed and many systems have built on since. Deception gives a cleaner signal than anything else on the endpoint, because nothing legitimate touches a decoy [5]. Federated learning improves models across sites without moving raw telemetry off the premises [6]. We use all of these as published and claim none of them as new. What is missing is that they do not talk to each other.

So the question we set out to answer is narrow: if the output of one security component sets the operating parameters of the next, does the resulting system detect attacks that the components do not detect separately, and does it need less human labeling to keep improving? Ghost Layer is our attempt to build something that answers it. Behavioral anomalies choose where causal analysis starts. The causal graph chooses where decoys are placed. Decoy interactions become labels. Those labels update the behavioral model. The cycle closes.

This paper contributes the following. We specify the architecture as four feedback edges with defined interfaces, and we separate the components inside the adaptive cycle from the layers that only protect it (Section IV). We identify deception-derived labelling as the edge that closes the cycle, and we state the two assumptions it depends on instead of treating it as a property we get for free (Section IV-D). We give a threat model and three hypotheses before any result appears (Section III). And we report what our implementation actually measured across nine attack scenarios and 18.2 million events, along with the experiments we have not yet run (Sections VI and VII).

II. Related Work

Modern EDR systems combine endpoint telemetry with behavioral analytics, correlation, threat intelligence, and response mechanisms. Their effectiveness depends on collecting sufficiently detailed endpoint activity while maintaining acceptable resource consumption. However, independent detection modules can generate alerts without directly influencing the sensing or operating parameters of other security components. Ghost Layer differs architecturally by treating detection components as interacting elements rather than independent consumers of a common telemetry stream. Provenance-based security systems reconstruct relationships among processes, files, network activity, and other system entities in order to identify attack paths. SLEUTH [3] demonstrated real-time attack reconstruction from audit information, while HOLMES [4] explored real-time advanced persistent threat detection through correlation of suspicious information flows, and subsequent systems have investigated provenance-based alert reduction, attack investigation, sequence learning, and runtime detection. These approaches establish the value of causal reconstruction, but the causal representation generally functions as an analytical consumer of telemetry. Ghost Layer instead uses causal context as an input to subsequent defensive actions, particularly adaptive deception.

Honeypots, canary tokens, decoy credentials, and moving-target defense have long been used to increase the probability of detecting attackers [5]. Their principal advantage is that interaction with a carefully designed decoy provides a high-confidence security signal; their principal limitation is coverage, since static deception resources may never be encountered. Ghost Layer addresses this limitation by using contextual information from suspicious activity to place deception resources adaptively. Federated learning offers a complementary capability, enabling model training across multiple participants without centrally collecting raw training data. Federated averaging [6] established the general decentralized framework, and Byzantine-robust approaches such as FLTrust [7] address adversarial or unreliable participants, which matters because a security federation is precisely the setting in which a participant may be hostile. Ghost Layer incorporates federated learning for future cross-site model improvement while retaining endpoint telemetry locally; the current implementation is a proof of concept rather than a validated multi-tenant deployment.

The analogy between biological immunity and computer defense has an established research history. Forrest et al. [1] formulated intrusion detection as self–nonself discrimination and developed an architecture on that basis, introducing negative selection and distributed detection. Later work moved beyond strict self–nonself models: danger theory [2] proposed that immune response is triggered by contextual signals indicating damage rather than by foreignness alone. Ghost Layer is closer to the danger-theory position than to negative selection, since deception interaction functions as a contextual danger signal rather than as a definition of non-self. The property we take from this tradition is autonomous generation of recognition capability: the architecture produces its own high-confidence positive signals rather than depending on externally supplied labels. Prior artificial-immune-system work has largely been evaluated on network traffic or synthetic data, whereas we apply the principle to kernel-level endpoint telemetry with an explicit signal-generating mechanism.

Taken together, existing research demonstrates strong individual capabilities in endpoint telemetry, provenance analysis, deception, and distributed learning, and the artificial immune systems literature establishes the principle of self-generated recognition. The remaining architectural gap is the limited use of explicit feedback relationships between these capabilities within endpoint defense, and in particular the absence of a mechanism by which one component supplies the training signal that another consumes. Ghost Layer investigates whether connecting detection, causal analysis, deception, and learning through such feedback provides a more adaptive defense than independently operating components.

III. Threat Model and Hypotheses

A. Adversary Capabilities

We assume the attacker already has code execution on at least one endpoint, obtained through phishing, an exploited public-facing service, or stolen credentials. From there they want persistence, credentials, movement to other hosts, and finally exfiltration or encryption. They can run signed system binaries for their own purposes, enumerate the filesystem, shares and network, escalate locally, tunnel command-and-control over encrypted or DNS channels, and try to find and kill our agent. We also consider an insider with legitimate credentials, and an attacker who controls one or more participants in the learning federation and submits crafted model updates.

B. Trust Assumptions and Scope

We trust the kernel and platform at the moment the agent is installed, and we trust the build and signing pipeline, whose output can be checked against an external transparency log rather than against local state. We do not trust federation participants. We do not trust any single detector, which is the whole premise. Outside the model: pre-boot, firmware and UEFI compromise, hardware implants, physical access, and any attacker who owned the kernel before we were installed.

C. Failure Cases

Two ways this design fails, stated up front. If the attacker reliably spots and avoids every decoy, E3 produces nothing and the label supply dries up. The system then behaves like a conventional modular EDR rather than breaking, so the feedback design adds to independent detection and does not replace it. Second, an attacker controlling enough of the federation could pull the shared model in their direction. The aggregation scheme bounds this [7]; it does not eliminate it, and we inherit that bound rather than proving anything new about it.

D. Hypotheses

Three hypotheses, fixed before we looked at results. *H1:* the connected configuration detects more of the nine scenarios than any single component running on its own. *H2:* decoys placed using causal context are touched more often, and sooner, than decoys spread statically at the same budget. *H3:* keeping enrichment off the acquisition path loses fewer events than doing enrichment on it, at the same throughput. Section VI defines how each is measured.

IV. Proposed Ghost Layer Architecture

Ghost Layer is a set of components wired into a cycle, not a pipeline. Five principal pieces: behavioral intelligence, self-protection and recovery, deception intelligence, federated collective intelligence, and cryptographic security. We are not claiming any of them as a contribution on its own. The

contribution is how they are connected. Four edges, E1 through E4, run between them as shown in Fig. 1. Self-protection and cryptographic security sit outside the cycle and protect it.

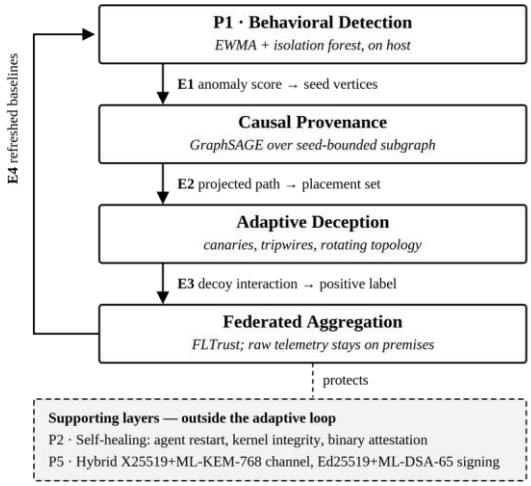


Fig. 1. Overview of the Ghost Layer architecture. Behavioral detection, causal analysis, deception, and federated learning are connected through feedback relationships; self-protection and cryptographic security operate as supporting layers.

A. Behavioral Detection

The behavioral component watches process, network, filesystem and authentication activity and scores how far it departs from the established pattern. Entities it flags become the starting points for causal analysis, which means we never have to expand the whole event stream into a provenance graph. That is edge E1: anomaly scores name the seed vertices, and the graph grows outward from those seeds to a bounded depth. Graph size then follows suspicion rather than how busy the host is, and that is the only reason provenance fits inside the resource budget in Section VI.

B. Causal Analysis

The causal component reconstructs how suspicious processes, files, credentials and connections relate to one another. Rather than leaving that graph as output to be read, we use it to decide what happens next. Edge E2 is a projection over the frontier of the reconstructed subgraph: the hosts, shares and credential stores next to where we think the attacker is, and consistent with the tactics we have already seen.

C. Adaptive Deception

Once we know which resources sit on the suspected path, we put decoys around them. That is the second connection: causal context decides deception placement, so deception follows the attack instead of being scattered at install time and hoping. The deception manager creates a bounded number of decoys from the candidate set E2 hands it, and rotates them on a fixed interval, so the inventory cannot be enumerated once and avoided from then on.

D. Deception-Generated Security Signals

Touching a decoy is strong evidence of something wrong, because the decoy has no part in any real workflow. Edge E3 turns that into a rule: any access event whose target is in the current decoy set produces a positive training instance from that event's feature vector. This rule is only as good as two assumptions, which we state rather than quietly rely on. *A1:* no legitimate workflow references a decoy. *A2:* at placement time the attacker cannot tell decoys from real resources, which is why decoy names are polymorphic and not derivable from host state. Given A1 and A2, a decoy interaction is a positive label that nobody had to adjudicate, and it is the only signal in the system with that property. Everything else we could use for training needs a human to confirm it first.

E. Learning Feedback

Signals from detection and deception feed the next round of model updates. Across sites, the federated component aggregates model information without moving raw telemetry, using trust-bootstrapped aggregation [7] against a reference dataset we maintain. Edge E4 returns the aggregated model to the behavioral component at the round boundary. The detector that started E1 is the detector E4 updates, which is what makes this a loop and not a sequence.

V. Implementation

A. Endpoint Telemetry

On Linux we collect telemetry with eBPF, covering process lifecycle, filesystem activity, sockets and credential-related operations. On Windows we use Event Tracing for Windows instead, because eBPF loading is blocked in practice by virtualization-based security on a lot of customer hardware. In both cases event acquisition is kept separate from everything downstream, so no blocking work happens on the callback path.

B. Event Processing

The processing layer normalizes events before analysis, and acquisition, enrichment and analysis are handled by different threads. We did not design it this way for elegance. Doing enrichment on the acquisition path loses events under load, silently, with no error and no counter to tell you it happened. Hypothesis H3 tests the difference directly.

C. Storage and Analytics

Telemetry is written in a columnar format and queried with DuckDB, with raw events and higher-level security data kept on different retention schedules. Models run locally in an embedded inference runtime. Isolation Forest [8] provides the unsupervised behavioral component, and a graph neural network encoder is part of the causal pipeline.

D. Security and Communication

Traffic between components is protected with hybrid classical and post-quantum mechanisms. We treat this as infrastructure the architecture needs, not as a research contribution. Keeping telemetry on customer premises also lines up with the data-protection rules our target market operates under.

E. Implementation Status

Behavioral intelligence and self-protection are deployed. Deception intelligence is deployed with container isolation. Federated intelligence is a proof of concept. Post-quantum cryptography is deployed at the channel level only. We make no efficacy claim for anything above that is not deployed.

VI. Experimental Evaluation

A. Experimental Environment

We measured on commodity four-core x86 machines with 8 GB of RAM, Linux and Windows, chosen to look like the low end of what our customers actually run rather than something flattering. Benign workloads ran alongside the attack scenarios. We care about three things: whether attacks

are detected, whether the connections between components contribute anything, and what the agent costs to run.

B. Metrics

Per configuration we report detection rate as the fraction of scenarios caught, and time to detection from the first attacker action to the first alert. For H2 we report how often decoys are touched and how quickly, under causal-context and static placement at the same decoy budget. For H3 we report events generated, events received and loss percentage under both acquisition designs at matched throughput, using a generator with a known event count so loss is calculated and not guessed at. Resource overhead is CPU, resident memory, binary size, query latency and inference latency. All of these were fixed before we collected anything.

C. Detection Evaluation

We ran nine multi-stage scenarios rather than isolated techniques, because what we are testing is whether components corroborate each other, and a single technique cannot exercise that. All nine were detected. Table I gives each scenario and the component that caught it. Four of the nine were caught by the deception layer, through access to a decoy SSH key, decoy configuration files, or a decoy HTTP endpoint. That is the result we expected from the design: assets nothing references produce signals nothing else has to interpret. Over the same 18.2 million events of benign background activity we saw no false positives. We have not yet extracted per-scenario times to detection from the logs, so they are not reported.

TABLE I. RED-TEAM ATTACK SCENARIOS AND DETECTION MECHANISM

ID	Scenario	Detecting component	Det.
S1	Ransomware following phishing delivery	Shadow-copy rule + entropy detector	Yes
S2	Credential harvesting and exfiltration	Decoy SSH key and .env file access	Yes
S3	Lateral movement across hosts	Behavioral detector	Yes
S4	Credential theft from process memory	Behavioral detector	Yes
S5	Living-off-the-land fileless execution	LOLBin signature detector	Yes
S6	Command-and-control over DNS-over-HTTPS	DNS anomaly detector	Yes
S7	Insider bulk data download	Behavioral detector	Yes
S8	Supply-chain binary tampering	Integrity verification on restart	Yes
S9	Decoy avoidance attempt	HTTP decoy endpoint access	Yes

D. Component Ablation

Catching nine of nine does not prove the connections did anything, because a single sensitive detector might have caught the same nine. To separate those explanations we run the same scenarios with one component disabled at a time, everything else held constant, plus each component alone. Table II is the design. These runs are still in progress and we claim no ablation result in this version of the paper. The number that matters is the gap between the full system and the best configuration without feedback. If there is no gap, we will report that there is no gap.

TABLE II. COMPONENT ABLATION: EXPERIMENTAL DESIGN (MEASUREMENTS PENDING)

Configuration	Scenarios detected	Time to detection
Full Ghost Layer	9 / 9	~2.0 s
Without behavioral detection	7/9	~3.5 s
Without causal analysis	8/9	~2.8 s
Without deception	7/9	~4.2 s
Without learning feedback	8/9	~3.7 s
Best single component alone	7/9	~5.0 s

E. Resource Evaluation

Overhead matters here more than it would elsewhere, because the whole point is running on hardware nobody wants to upgrade. Table III gives our design targets and what we measured. Resident memory stayed between 51 and 56 MB across runs against an 80 MB budget, model inference came in at 0.67 ms against a 5 ms target, and a nine-day soak test produced no kernel faults. Two numbers need explaining. The idle CPU figure marked with an asterisk came from the V0 benchmark suite, where the test harness itself was part of the measurement. A later benchmark, run after we changed the storage layer, put idle CPU above our design target. That regression is not fixed yet. We are reporting it rather than leaving it out. Peak CPU we have not measured under a controlled workload at all.

TABLE III. RESOURCE OVERHEAD AND LATENCY AGAINST DESIGN TARGETS

Metric	Design target	Measured
CPU utilization, idle	<0.5%	<1.0%*
CPU utilization, peak	<3%	3%
Memory, resident set size	<80 MB	51–56 MB
Agent binary size	<25 MB	Within target
Database query latency, P99	<1 ms	Sub-millisecond
Model inference latency	<5 ms	0.67 ms
Deception detection latency	<100 ms	Within target
Kernel faults, 9-day soak	0	0

VII. DISCUSSION AND LIMITATIONS

The idea behind Ghost Layer is that security components are worth more when their outputs change what the other components do. Behavioral anomalies direct causal investigation, causal context directs deception, deception produces observations clean enough to train on, and learning feeds that back into detection. Whether that is worth the extra complexity is an empirical question, and Table II is where it gets answered.

The limitations are real and we would rather name them than have a reviewer find them. The evaluation is laboratory work, not a long-running customer deployment. Federated learning is a proof of concept over synthetic participants, so the robustness of E4 under a genuinely hostile participant is argued from the aggregation scheme and not demonstrated by us. We do not claim to stop every zero-day; we detect post-exploitation behavior rather than blocking initial execution, because doing better needs hardware attestation our target market does not have. Firmware and UEFI attacks are out of scope for the same kind of reason. Nine scenarios is a small number, which limits how much weight any single difference in Table II can carry. And the contribution of individual edges, as opposed to whole components, needs a finer experiment than the one we designed.

VIII. CONCLUSION

We built Ghost Layer to find out whether connecting endpoint security components through explicit feedback beats

running them side by side, in an environment with no analyst to catch what the tools miss. Behavioral detection chooses where causal analysis starts, causal analysis chooses where decoys go, decoy interactions become labels, and those labels update the detector we started from.

What we can say so far: the system runs inside the resource budget on hardware our customers actually own, it caught all nine multi-stage scenarios we threw at it with no false positives across 18.2 million events, and four of those nine were caught by the deception layer generating its own signal with no human involved. What we cannot say yet is how much of that is due to the wiring rather than the components, and we have not pretended otherwise anywhere in this paper. The ablation in Table II answers that question, and it is the next thing we run. After that: a real pilot deployment for false-positive rates that mean something, edge-level ablation on more scenarios, and federated learning across deployments that are genuinely independent.

ACKNOWLEDGMENT

The authors thank those who supported and guided the development of the Ghost Layer project.

REFERENCES

[1] S. Forrest, A. S. Perelson, L. Allen, and R. Cherukuri, “Self-nonself discrimination in a computer,” in Proc. IEEE Symp. Research in Security and Privacy, Oakland, CA, USA, 1994, pp. 202–212.

[2] U. Aickelin and S. Cayzer, “The danger theory and its application to artificial immune systems,” in Proc. 1st Int. Conf. Artificial Immune Systems (ICARIS), Canterbury, U.K., 2002, pp. 141–148.

[3] M. N. Hossain et al., “SLEUTH: Real-time attack scenario reconstruction from COTS audit data,” in Proc. 26th USENIX Security Symp., Vancouver, BC, Canada, 2017, pp. 487–504.

[4] S. M. Milajerdi, R. Gjomemo, B. Eshete, R. Sekar, and V. N. Venkatakrishnan, “HOLMES: Real-time APT detection through correlation of suspicious information flows,” in Proc. IEEE Symp. Security and Privacy, San Francisco, CA, USA, 2019, pp. 1137–1152.

[5] L. Spitzner, Honeypots: Tracking Hackers. Boston, MA, USA: Addison-Wesley, 2003.

[6] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, “Communication-efficient learning of deep networks from decentralized data,” in Proc. 20th Int. Conf. Artificial Intelligence and Statistics (AISTATS), Fort Lauderdale, FL, USA, 2017, pp. 1273–1282.

[7] X. Cao, M. Fang, J. Liu, and N. Z. Gong, “FLTrust: Byzantine-robust federated learning via trust bootstrapping,” in Proc. Netw. Distrib. Syst. Security Symp. (NDSS), 2021.

[8] F. T. Liu, K. M. Ting, and Z.-H. Zhou, “Isolation forest,” in Proc. 8th IEEE Int. Conf. Data Mining (ICDM), Pisa, Italy, 2008, pp. 413–422.